

课程笔记

MoE 初步：参数量计算、Qwen3-30B-A3B、 GQA 与 Sparse MoE

基于公开课程资料整理

2025

LLM 架构 MoE Qwen3-30B-A3B 参数量计算

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 15 分钟

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 引言：为什么关注 MoE ?	2
2 Dense vs. MoE: 配置对比	2
2.1 模型配置一览	2
2.2 架构演进脉络	2
3 GQA (Grouped Query Attention) 详解	3
3.1 GQA 的动机与机制	3
3.2 $d_{\text{model}} \neq n_{\text{heads}} \times d_{\text{head}}$	3
3.3 本章小结	3
4 Sparse MoE 架构详解	3
4.1 MoE 层的结构	3
4.2 Router 机制	4
4.3 Expert 的 FFN 结构	4
4.4 MoE 的最终输出	4
4.5 本章小结	4
5 参数量计算	5
5.1 Token Embedding	5
5.2 每层参数量	5
5.3 总参数量与激活参数量	5
5.4 本章小结	5
6 总结与延伸	6
6.1 拓展阅读	6

1 引言：为什么关注 MoE？

本期是「大语言模型架构」系列的第一讲。近年来，大模型架构的演进日新月异，其中 **Mixture of Experts (MoE)** 架构因其在推理效率上的巨大优势而备受瞩目。

以 DeepSeek R1 为例：它是一个约 670B 总参数的 MoE 模型，但推理时仅激活约 72B 的参数。与一个完整的 72B Dense 模型相比，两者的推理速度基本一致——这意味着 MoE 模型在保持巨大模型容量（能力强）的同时，推理效率极高（激活参数少）。

MoE 的核心价值

- 总参数量大 $\Rightarrow$ 模型容量大、能力强
- 激活参数量小 $\Rightarrow$ 推理速度快、效率高
- Qwen2.5 时代还是全 Dense 模型，到 Qwen3 已大量采用 MoE 变体

本期以 **Qwen3-32B (Dense)** 和 **Qwen3-30B-A3B (MoE)** 为对象，通过参数量的对比计算来理解 MoE 架构。

2 Dense vs. MoE：配置对比

2.1 模型配置一览

表 1: Qwen3-32B (Dense) vs. Qwen3-30B-A3B (MoE) 配置对比

配置项	Qwen3-32B (Dense)	Qwen3-30B-A3B (MoE)
层数 (Layers)	64	48
模型维度 d_{model}	5120	2048
注意力头数	64	32
Head Dimension	128	128
KV 组数 (GQA)	8	4
FFN 中间维度	25600	768 (每个 Expert)
Expert 总数	—	128
激活 Expert 数	—	8
Tie Word Embeddings	False	False

为什么 MoE 模型「更窄更浅」？

从 Dense 的 64 层 $\rightarrow$ MoE 的 48 层， d_{model} 从 5120 $\rightarrow$ 2048，注意力头数从 64 $\rightarrow$ 32。MoE 模型通过大量专家（128 个）来补偿这些维度上的缩减，总参数量仍然很大（ $\sim 30\text{B}$ ），但每次推理只激活 8 个专家，对应的激活参数量仅约 3.3B。

2.2 架构演进脉络

当前主流 LLM 架构大多脱胎于 Transformer $\rightarrow$ LLaMA 2 的技术路线：

- **RMSNorm**：替代 LayerNorm，去掉均值中心化，只做缩放

- **SwiGLU**: 替代 ReLU 的 FFN 激活函数
- **RoPE**: 旋转位置编码
- **GQA**: 分组查询注意力, 压缩 KV Cache

MoE 架构的核心改动: 将每层的 **FFN 模块替换为 Sparse MoE 模块**, 即从一个共享的两层 MLP 变成 128 个独立的小 FFN (Expert), 每次只路由到 top-8 个 Expert。

3 GQA (Grouped Query Attention) 详解

3.1 GQA 的动机与机制

GQA 核心思想

标准 MHA 中每个 Query 头对应一个独立的 KV 头。GQA 让多个 Query 头共享同一组 KV 头, 从而大幅压缩 KV Cache 的存储开销。

以 Qwen3-30B-A3B 为例:

- 32 个 Query 头, 4 组 KV 头
- 每 $32/4 = 8$ 个 Query 头共享一组 K, V
- Query 头 0-7 $\rightarrow$ KV 组 0; Query 头 8-15 $\rightarrow$ KV 组 1; 以此类推

3.2 $d_{\text{model}} \neq n_{\text{heads}} \times d_{\text{head}}$

维度不一定匹配

在 LLaMA 2 或 GPT-2 中, $d_{\text{model}} = n_{\text{heads}} \times d_{\text{head}}$ 。但 Qwen3 的 Dense 模型中 $d_{\text{model}} = 5120$, 而 $n_{\text{heads}} \times d_{\text{head}} = 64 \times 128 = 8192 \neq 5120$ 。这意味着 W_Q 不再是方阵, 而是 5120×8192 的非方阵。

3.3 本章小结

GQA 通过让多个 Query 头共享 KV 组来压缩 KV Cache, 是当前主流 LLM 的标配。Qwen3 进一步打破了 d_{model} 与头维度的对齐约束, 使用非方阵投影矩阵。

4 Sparse MoE 架构详解

4.1 MoE 层的结构

每个 MoE 层包含以下组件:

1. **Router (路由器)**: 将 d_{model} 维的 token embedding 映射到 128 维的 logits 向量
2. **Top-K 选择**: 从 128 个 Expert 中选出 top-8
3. **Softmax Gating**: 对 top-8 的 logits 做 softmax, 得到加权系数

4. **Expert FFN**: 每个 Expert 是一个两层 MLP ($W_{\text{up}}, W_{\text{gate}}, W_{\text{down}}$)
5. **加权求和**: 将 8 个 Expert 的输出按 gating 系数线性加权求和

4.2 Router 机制

$$\text{Router} : \mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{N_{\text{experts}}}$$

即 $W_{\text{router}} \in \mathbb{R}^{2048 \times 128}$, 将每个 token 的 embedding 映射到 128 个 Expert 对应的分数。

选出 top-8 后, 对这 8 个分数做 softmax (由配置参数 `norm_topk_prob` 控制, 默认为 True), 得到归一化的 gating 系数 $g_1, g_2, \dots, g_8$ 。

4.3 Expert 的 FFN 结构

每个 Expert 是一个带 SwiGLU 的两层 MLP:

$$\text{Expert}(x) = W_{\text{down}} \cdot [\text{SiLU}(W_{\text{up}} \cdot x) \odot (W_{\text{gate}} \cdot x)]$$

其中:

- $W_{\text{up}} \in \mathbb{R}^{2048 \times 768}$: 将 d_{model} 映射到中间维度
- $W_{\text{gate}} \in \mathbb{R}^{2048 \times 768}$: 门控分支, 与 W_{up} 的输出逐元素相乘
- $W_{\text{down}} \in \mathbb{R}^{768 \times 2048}$: 将中间维度映射回 d_{model}

Dense FFN vs. MoE Expert FFN

Dense 模型的 FFN 中间维度为 $25600 = 5 \times d_{\text{model}}$, 而 MoE 每个 Expert 的中间维度仅为 768。虽然单个 Expert 远小于 Dense FFN, 但 128 个 Expert 总参数量为 $128 \times 3 \times 2048 \times 768 \approx 604\text{M}$ (仅 FFN 部分), 再加上其他层, 总参数量约 30B。

4.4 MoE 的最终输出

$$\text{MoE}(x) = \sum_{i=1}^8 g_i \cdot \text{Expert}_i(x)$$

每个 Expert 输出一个 2048 维的向量, 通过 gating 系数加权求和, 得到最终的 MoE 层输出。

4.5 本章小结

Sparse MoE 的核心思路是「路由 + 稀疏激活」: 用 Router 为每个 token 选择最相关的 top-K 个 Expert, 其余 Expert 不参与计算。这使得模型总参数量很大但每次推理的计算量很小。

5 参数量计算

5.1 Token Embedding

Qwen3-30B-A3B 的 `tie_word_embeddings = False`，即输入和输出 embedding 不共享，需要两份：

$$P_{\text{embed}} = 2 \times V \times d_{\text{model}}$$

其中 V 是词表大小， $d_{\text{model}} = 2048$ 。

5.2 每层参数量

每层包含三个部分：

1. GQA 注意力部分：

$$P_{\text{attn}} = d_{\text{model}} \times (n_Q \cdot d_h) + 2 \times d_{\text{model}} \times (n_{KV} \cdot d_h) + (n_Q \cdot d_h) \times d_{\text{model}}$$

其中 $n_Q = 32, n_{KV} = 4, d_h = 128$ ：

$$P_{\text{attn}} = 2048 \times 4096 + 2 \times 2048 \times 512 + 4096 \times 2048$$

2. Router：

$$P_{\text{router}} = d_{\text{model}} \times N_{\text{experts}} = 2048 \times 128$$

3. MoE Expert (全部 128 个 Expert)：

$$P_{\text{MoE}} = N_{\text{experts}} \times 3 \times d_{\text{model}} \times d_{\text{ffn}} = 128 \times 3 \times 2048 \times 768$$

5.3 总参数量与激活参数量

30B vs. 3.3B 的由来

- **总参数量** ($\sim 30.5\text{B}$)：Token Embedding + 48 层 $\times$ (Attention + Router + **128** $\times$ Expert FFN)
- **激活参数量** ($\sim 3.3\text{B}$)：Token Embedding + 48 层 $\times$ (Attention + Router + **8** $\times$ Expert FFN)
- 两者唯一的区别：Expert 数从 128 $\rightarrow$ 8

这正是模型名称「30B-A3B」的含义：总参数约 30B，激活参数约 3B。

5.4 本章小结

通过逐项计算，我们清楚地看到 MoE 模型的参数量如何分配：大量参数存储在 128 个 Expert 中，但推理时只激活 8 个。这实现了「大容量、低计算」的目标。

6 总结与延伸

本期从 Qwen3 的 Dense 和 MoE 两个变体入手，通过配置对比和参数量逐项计算，建立了对 Sparse MoE 架构的直观理解：

1. MoE 将 FFN 替换为多个独立 Expert + Router，实现稀疏激活
2. GQA 通过共享 KV 头压缩 KV Cache，是当前 LLM 的标配
3. Qwen3 打破了传统的 $d_{\text{model}} = n_{\text{heads}} \times d_{\text{head}}$ 约束
4. 30B 总参数中，推理时仅激活约 3.3B，推理效率接近同规模 Dense 模型

6.1 拓展阅读

- Sebastien Raschka 的博客：LLM 架构详解系列
- Qwen3 官方技术报告
- DeepSeek V2/V3 技术报告中的 MoE 设计
- 「Post NOS Chat GPT」系列中关于 LLaMA 2、GQA 的讲解